

Autonomous AI Trust Fabric

for Clinical Decision Intelligence

Product Requirements Document — v1.0 — Complete Zero-Assumption Build Guide

For: AI coding assistant (Claude Code, Cursor, Windsurf, or similar) · Domain: Healthcare AI Governance / Clinical Decision Support

Layer	Technology
Frontend	React 18 + TypeScript + Vite + Tailwind CSS
Backend	FastAPI (Python 3.11)
Agent Orchestration	LangGraph 0.1.9 + LangChain 0.2.3
LLM (Explainability only)	OpenRouter — openai/gpt-oss-20b:free (default), swappable to openai/gpt-4o
RAG Embeddings	sentence-transformers (all-MiniLM-L6-v2), local, no external API
Storage	PostgreSQL 16 (Docker) — twin store + audit log
Scheduling	APScheduler (in-process) — Trust Monitoring job
Auth	Username + password → role lookup. No JWT, no Keycloak (phase 1)

1. Project Overview

1.1 Purpose

This is the single authoritative specification for the Autonomous AI Trust Fabric v1.0. A coding assistant must be able to build every file — frontend, backend, database schema, LangGraph agents, RAG knowledge base, tests, and scripts — without making assumptions. Every endpoint, schema, agent, and UI component required for implementation is described in this document.

1.2 What This System Does

Hospitals already use diagnostic AI models (vendor-built, black-box) to support clinical decisions — e.g., a model that reviews a chest X-ray and predicts "Pneumonia, 94% confidence." Today, that prediction is just a number and a label, with no structured, retrievable record of what data produced it, whether the model version was approved, why the prediction is clinically reasonable, or whether the model is behaving normally over time.

This system sits AFTER the diagnostic model and automatically builds a Digital Compliance Twin for every prediction — a permanent record combining data lineage, a RAG-grounded plain-English explanation, and a compliance check — so any prediction can be explained and justified at any time. The diagnostic model itself is never built, trained, or modified; it is treated as an external black box and its output is simulated with mock prediction records that follow the exact schema a real model would produce.

1.3 End-to-End System Flow (with RAG-grounded Explainability)

1. A prediction record + patient context arrive at POST /predictions (in production this would come from a real diagnostic model; in this phase it comes from data/predictions.json + data/patients.json).
2. FastAPI validates both payloads against Pydantic schemas (422 on failure).
3. The LangGraph pipeline runs synchronously, four nodes in strict sequence:
 - a. lineage_agent -- records source data + model version

- b. `compliance_agent` -- checks rulebook, sets `flagged` + `flag_reasons`
 - c. `explainability_agent` -- retrieves grounding facts from the local Knowledge Base, then calls the LLM (OpenRouter) constrained to ONLY use those facts
 - d. `twin_assembler_agent` -- merges all of the above into one Digital Compliance Twin record
4. The twin is written to the `digital_twins` table + an `audit_log` entry.
 5. If `flagged` = `true`, the twin is also visible in the Human Review Queue (a filtered view of `digital_twins`, NOT a separate table or agent).
 6. `POST /predictions` returns 201 with the full twin record.
 7. The Compliance/Governance user reviews flagged twins in the UI and Approves or Overrides (`POST /twins/{id}/review`).
 8. A separate, unrelated in-process job (APScheduler, NOT part of the LangGraph pipeline) runs the Trust Monitoring check periodically, scanning stored twins for confidence drift and rising flag rate, writing rows to `drift_alerts` when thresholds are crossed.
 9. The Dashboard, Twin Detail, Rulebook Settings, and Trust Monitoring pages read all of the above through the REST API described in Section 10.

1.4 Explicit Exclusions — Do Not Add These

These were deliberately scoped out. Adding them contradicts this specification:

- No Apache Kafka, no RabbitMQ, no message queue of any kind — predictions are ingested via direct synchronous HTTP POST
- No Apache Airflow — the single scheduled job (Trust Monitoring) runs on APScheduler inside the FastAPI process
- No Keycloak, no JWT, no OAuth — authentication is a hardcoded username/password → role lookup (see Section 14)
- No cloud hosting (AWS/Azure/GCP) — everything runs on localhost
- No real diagnostic model, no CNN, no model training of any kind — the diagnostic model is an external black box; its output is mock JSON
- No real FHIR/EHR integration
- No SHAP/LIME — the model is a black box with no accessible internals; explanations are RAG-grounded LLM narratives, never mathematical feature attributions
- No Prometheus/Grafana — LangSmith-style observability is out of scope for phase 1; standard Python logging is sufficient

2. Business Problem

If a prediction is later questioned — by a doctor, a hospital audit, or a regulator — the answer today is: dig through scattered logs and hope they still exist. Specifically, nobody can currently answer, for any single past prediction:

- What exact patient data produced this prediction?
- Which model version made the call, and was it an approved version?
- Why is the prediction clinically reasonable?
- Is the model behaving normally, or has its behavior drifted over time?

Example scenario: a hospital's existing pneumonia-detection model reviews a patient's chest X-ray and predicts "Pneumonia — 94% confidence." The doctor orders treatment. Three months later, a hospital audit team asks why the AI flagged pneumonia for this patient and what data influenced the decision. Today, that specific prediction moment was never recorded in a structured, retrievable way — the team must manually reconstruct it from scattered logs, if those logs even still exist. This system closes that gap by wrapping a permanent, structured trust record around every prediction the moment it is made.

3. Objectives

- Produce a Digital Compliance Twin for every diagnostic prediction, combining lineage, compliance status, and a grounded explanation
- Guarantee that every high-risk or low-confidence or non-approved-version prediction is routed to a human reviewer before being treated as final
- Prevent hallucinated clinical claims in the explanation layer by grounding all LLM output in a curated, retrievable knowledge base (RAG)
- Detect model drift (confidence decline, rising flag rate) automatically and surface it to Compliance/Governance users
- Keep the diagnostic model fully decoupled and swappable — the Trust Fabric never depends on model internals, only on a fixed JSON prediction schema
- Keep infrastructure proportional to the problem: no distributed systems, no cloud dependency, deployable in minutes on a single machine

4. Users and Roles

Two roles only. There is no third "admin" role and no per-organization tenancy in phase 1.

Role	Relationship to system	Pages visible
Clinician	Passive beneficiary — sees prediction + explanation alongside their existing workflow. Cannot approve/override.	Dashboard (read-only), Twin Detail (read-only)
Compliance/Governance	Active user — reviews flagged twins, edits the rulebook, monitors drift. Combines compliance officer, hospital AI governance, and IT admin duties into one role.	Dashboard (full), Twin Detail (with Approve/Override), Rulebook Settings, Trust Monitoring

4.1 Demo Login Table (hardcoded — see Section 14)

Username	Password	Role	Display Name
dr.mitchell	clinician123	clinician	Dr. Sarah Mitchell
compliance.lead	compliance123	compliance_governance	Compliance Team

5. Complete Technology Stack

5.1 Backend — requirements.txt

Package	Version	Purpose
fastapi	0.111.0	REST API framework
uvicorn[standard]	0.30.1	ASGI server
pydantic	2.7.1	Request/response schema validation
python-multipart	0.0.9	Form/file parsing (future file upload support)
sqlalchemy	2.0.30	ORM for all Postgres tables
psycopg2-binary	2.9.9	Postgres driver
alembic	1.13.1	Schema migrations
langgraph	0.1.9	Multi-agent state machine
langchain	0.2.3	LLM integration, prompt templates
langchain-openai	0.1.8	ChatOpenAI-compatible client, pointed at OpenRouter
openai	1.30.0	OpenAI-compatible SDK (used with OpenRouter base_url)
sentence-transformers	2.7.0	Local embedding model for RAG retrieval
numpy	1.26.4	Cosine similarity computation for retrieval
apscheduler	3.10.4	In-process scheduled job for Trust Monitoring
python-dotenv	1.0.1	.env loading
pyyaml	6.0.1	config.yaml loading
pytest	8.2.0	Unit/integration tests
httpx	0.27.0	Test client for FastAPI

5.2 Frontend — frontend/package.json

Package	Version	Purpose
react	18.3.0	UI library
react-dom	18.3.0	DOM rendering
typescript	5.4.0	Type-safe language
vite	5.2.0	Build tool and dev server
tailwindcss	3.4.0	Utility CSS framework
react-router-dom	6.22.0	Client-side routing
recharts	2.12.0	Confidence trend / flag-rate trend charts
lucide-react	0.383.0	Icon library

@types/react	18.3.0	TypeScript types
autoprefixer	10.4.0	PostCSS autoprefixer
postcss	8.4.0	CSS processing

5.3 llm_config.py — LLM Client Configuration

File: backend/llm_config.py (imported ONLY by explainability_agent.py — no other agent may instantiate an LLM client).

```
from langchain_openai import ChatOpenAI
import os

llm_explainability = ChatOpenAI(
    model=os.environ.get('EXPLAINABILITY_MODEL', 'openai/gpt-oss-20b:free'),
    base_url='https://openrouter.ai/api/v1',
    api_key=os.environ['OPENROUTER_API_KEY'],
    temperature=0.2,
    max_tokens=400,
)
# To use GPT-4o instead: set EXPLAINABILITY_MODEL=openai/gpt-4o in .env
```

6. Complete Project Folder Structure

```
trust_fabric/
.env                      # secrets -- never commit
.env.example
.gitignore               # excludes .env, __pycache__/, node_modules/
docker-compose.yml       # Postgres only
requirements.txt
config.yaml              # rulebook defaults, drift thresholds
README.md

data/
knowledge_base.json      # curated clinical facts for RAG
patients.json            # seed data (60 patients)
predictions.json         # seed data (60 predictions)
compliance_rulebook.json # seed data (initial rulebook)

backend/
__init__.py
main.py                  # FastAPI app, CORS, router mounting, scheduler startup
llm_config.py            # OpenRouter client -- explainability_agent only
models/
__init__.py
schemas.py              # ALL Pydantic request/response models
db/
__init__.py
session.py              # SQLAlchemy engine + session factory
models.py               # ORM table definitions (Section 8)
routers/
__init__.py
auth.py                 # /auth/*
predictions.py          # POST /predictions
twins.py                # /twins/*
rulebook.py             # /rulebook/*
trust_monitoring.py     # /trust-monitoring/*
patients.py             # /patients/*
dashboard.py            # /dashboard/summary
services/
__init__.py
auth_service.py         # hardcoded user table + role lookup
audit_service.py        # append-only audit_log writer
rulebook_service.py     # rulebook read/write + changelog
drift_service.py        # confidence/flag-rate drift computation
rag_service.py          # KB loading, embedding, retrieval
agents/
__init__.py
state.py                # AgentState TypedDict
graph.py                # build_graph(), run_trust_fabric_pipeline()
lineage_agent.py
compliance_agent.py
explainability_agent.py
twin_assembler_agent.py
trust_monitoring/
__init__.py
monitor.py              # APScheduler job -- NOT a LangGraph node

scripts/
init_db.py              # creates all tables
seed_data.py            # loads data/*.json into Postgres
run_pipeline_on_seed_data.py # POSTs every seed prediction through the API

tests/
test_lineage_agent.py
test_compliance_agent.py
test_explainability_agent.py
test_twin_assembler_agent.py
```



```
test_rulebook_rules.py
test_drift_service.py
test_api_predictions.py
test_api_twins.py
fixtures/
  sample_prediction.json | sample_patient.json | sample_twin.json

frontend/
package.json | tsconfig.json | vite.config.ts | tailwind.config.ts
postcss.config.js | index.html
.env.local          # VITE_API_URL=http://localhost:8000
src/
  main.tsx | App.tsx | index.css
  contexts/AuthContext.tsx    # in-memory user + role, no token
  pages/
    Login.tsx
    Dashboard.tsx
    TwinDetail.tsx
    RulebookSettings.tsx
    TrustMonitoring.tsx
  components/
    layout/Sidebar.tsx | Topbar.tsx | ProtectedRoute.tsx
    ui/StatusBadge.tsx | MetricCard.tsx | DriftAlertBanner.tsx
    ReviewActionPanel.tsx | ChangeLogTable.tsx | TagInput.tsx
    charts/ConfidenceTrendChart.tsx | FlagRateTrendChart.tsx
  lib/api.ts | types.ts | utils.ts
```

7. Backend Architecture

7.1 backend/main.py

Creates the FastAPI app. Mounts CORS middleware (allow_origins=["http://localhost:5173"]). Includes all routers from backend/routers/. On startup event: (1) calls db.session.init_engine() to verify the Postgres connection, (2) loads and embeds the RAG knowledge base once via rag_service.load_knowledge_base(), caching embeddings in a module-level variable, (3) starts the APScheduler job from trust_monitoring/monitor.py on a configurable interval (default: every 6 hours, matching the UI's "Auto-checks enabled every 6 hours" footer text).

7.2 Synchronous Pipeline Execution

POST /predictions invokes the LangGraph pipeline synchronously (not via a background task or queue) and returns the completed twin in the same HTTP response. This is an explicit, documented scope decision: at the data volumes this system handles (tens to low hundreds of predictions), synchronous processing is simpler to reason about, easier to test, and avoids introducing a queue or polling endpoint that the demo does not need. If throughput requirements grow, this is the first place to introduce a background task queue — see Section 20.

7.3 Services Layer Responsibilities

Service	Responsibility
auth_service.py	Hardcoded USERS dict; authenticate(username, password) -> user dict or None
audit_service.py	write_audit_entry(entity_type, entity_id, action, actor, details) -- appends to audit_log table, never updates or deletes
rulebook_service.py	get_rulebook(), update_rulebook(new_rulebook, changed_by) -- writes changelog row on every field that changed
drift_service.py	compute_drift(model_version, window_days) -- returns confidence trend series, flag-rate trend series, and any newly crossed thresholds
rag_service.py	load_knowledge_base(), retrieve(prediction_label, symptoms, top_k=2) -- returns ranked KB entries + similarity scores

8. Database Schema (PostgreSQL)

All tables defined as SQLAlchemy ORM models in backend/db/models.py. Primary keys are UUID strings generated with uuid4() unless noted otherwise.

8.1 patients

Column	Type	Notes
patient_id	VARCHAR PK	e.g. 'P0009' -- matches data/patients.json
age	INTEGER	
symptoms	JSONB	list of strings
scan_type	VARCHAR	e.g. 'chest_xray'
relevant_history	TEXT	

8.2 predictions

Column	Type	Notes
id	UUID PK	
patient_id	VARCHAR FK -> patients.patient_id	
model_name	VARCHAR	e.g. 'vit-xray-pneumonia-classification'
model_version	VARCHAR	e.g. 'v2.1'
prediction	VARCHAR	e.g. 'PNEUMONIA', 'NORMAL'
confidence	FLOAT	0.0 - 1.0
timestamp	TIMESTAMPTZ	when the (mock) model produced this prediction
received_at	TIMESTAMPTZ	when POST /predictions was called, server-set

8.3 digital_twins

Column	Type	Notes
twin_id	UUID PK	
patient_id	VARCHAR FK -> patients.patient_id	
prediction	JSONB	{label, confidence, model_version}
lineage	JSONB	{source, input_captured_at, patient_id, model_version}
compliance	JSONB	{model_version_approved, confidence_ok, high_risk_requires_review, flagged, flag_reasons: []}
explanation	TEXT	the LLM-generated, RAG-grounded narrative
explanation_type	VARCHAR	'narrative_llm_rag_grounding' (see Section 11.3.c)

grounded_in_sources	JSONB	list of {id, source, similarity_score} from the KB
low_grounding_confidence	BOOLEAN	true if best KB match similarity < 0.40
flagged	BOOLEAN	denormalized copy of compliance.flagged, indexed for fast Dashboard filtering
twin_created_at	TIMESTAMPTZ	
review	JSONB, nullable	{reviewed_by, decision: 'approve' 'override', notes, reviewed_at} -- null until reviewed

8.4 rulebook

Single-row table (id is always 1) representing the CURRENT rulebook. Historical values live in rulebook_changelog, not as multiple rows here.

Column	Type	Notes
id	INTEGER PK	always 1
approved_model_versions	JSONB	list of strings, e.g. ['v2.0','v2.1']
minimum_confidence_threshold	FLOAT	e.g. 0.75
high_risk_conditions_requiring_review	JSONB	list of strings, e.g. ['Pneumonia','Tumor','Cancer']
updated_at	TIMESTAMPTZ	

8.4.1 rulebook_changelog

Column	Type	Notes
id	SERIAL PK	
changed_by	VARCHAR	username of the Compliance/Governance user
change_description	TEXT	human-readable, e.g. "Changed confidence threshold 0.70 -> 0.75"
changed_at	TIMESTAMPTZ	

8.5 drift_alerts

Column	Type	Notes
id	UUID PK	
model_version	VARCHAR	
alert_type	VARCHAR	'confidence_drop' 'flag_rate_rise' 'freshness_ok'
message	TEXT	e.g. 'Confidence dropped 15% this week for v2.1'
severity	VARCHAR	'High' 'Low'
status	VARCHAR	'Active' 'Cleared'
detected_at	TIMESTAMPTZ	

8.6 audit_log

Append-only. No UPDATE or DELETE statement may ever target this table from application code.

Column	Type	Notes
id	SERIAL PK	
entity_type	VARCHAR	'twin' 'rulebook' 'review'
entity_id	VARCHAR	
action	VARCHAR	e.g. 'twin_created', 'rulebook_updated', 'twin_reviewed'
actor	VARCHAR	username, or 'system' for automated actions
details	JSONB	action-specific payload
created_at	TIMESTAMPTZ	

9. Canonical Data Schemas (Pydantic, backend/models/schemas.py)

These are the exact JSON contracts. They must match data/patients.json, data/predictions.json, and data/compliance_rulebook.json byte-for-byte in field names and types.

9.1 PatientContext

```
class PatientContext(BaseModel):
    patient_id: str
    age: int
    symptoms: list[str]
    scan_type: str
    relevant_history: str
```

9.2 PredictionInput

```
class PredictionInput(BaseModel):
    patient_id: str
    model_name: str
    model_version: str
    prediction: str          # e.g. 'PNEUMONIA', 'NORMAL'
    confidence: float = Field(ge=0.0, le=1.0)
    timestamp: datetime

# On validation failure: HTTP 422 with FastAPI's default error body.
# On patient_id not found in `patients` table: HTTP 404
# {"detail": "Unknown patient_id: <id>. Patient context must be seeded first."}
```

9.3 ComplianceRulebook

```
class ComplianceRulebook(BaseModel):
    approved_model_versions: list[str]
    minimum_confidence_threshold: float = Field(ge=0.0, le=1.0)
    high_risk_conditions_requiring_review: list[str]
```

9.4 DigitalTwin (response schema)

```
class ComplianceResult(BaseModel):
    model_version_approved: bool
    confidence_ok: bool
    high_risk_requires_review: bool
    flagged: bool
    flag_reasons: list[str]

class GroundedSource(BaseModel):
    id: str
    source: str
    similarity_score: float

class ReviewRecord(BaseModel):
    reviewed_by: str
    decision: Literal['approve', 'override']
    notes: str
    reviewed_at: datetime

class DigitalTwin(BaseModel):
    twin_id: str
    patient_id: str
    prediction: dict          # {label, confidence, model_version}
    lineage: dict             # {source, input_captured_at, patient_id, model_version}
```

```
compliance: ComplianceResult
explanation: str
explanation_type: Literal['narrative_llm_rag_grounded']
grounded_in_sources: list[GroundedSource]
low_grounding_confidence: bool
flagged: bool
twin_created_at: datetime
review: ReviewRecord | None
```

10. Complete REST API Reference

Every endpoint the frontend calls. All request/response field names are exact.

10.1 Auth

POST /auth/login

Trigger: user clicks Sign In on the Login page.

```
Request: {"username": str, "password": str}
Response 200: {"success": true, "user": {"username": str, "role": "clinician"|"compliance_governance", "display_name": str}}
Response 401: {"success": false, "error": "Invalid credentials"}
```

Notes: no token is issued. The frontend stores the returned user object in AuthContext (React state) and sessionStorage. Every subsequent request that needs role-based authorization sends an X-User-Role header set from this stored value (see Section 14 for the honest limitations of this approach).

10.2 Predictions Ingestion

POST /predictions

Trigger: a (mock) diagnostic model produces a new prediction. Runs the full LangGraph pipeline synchronously.

```
Request: PredictionInput (Section 9.2)
Response 201: DigitalTwin (Section 9.4) -- the fully assembled twin
Response 404: {"detail": "Unknown patient_id: ..."}
Response 422: FastAPI validation error body
```

10.3 Twins

GET /twins

Trigger: Dashboard page load, and every filter/search/sort/page change.

```
Query params: status ('all'|"cleared"|"flagged", default 'all'),
              model_version (optional string),
              search (optional, matches patient_id substring),
              sort_by ('confidence'|"twin_created_at", default 'twin_created_at'),
              sort_dir ('asc'|"desc", default 'desc'),
              page (default 1), limit (default 20)
Response 200: {
  "total": int, "page": int, "limit": int,
  "twins": [ { "twin_id", "patient_id", "prediction": {label, confidence, model_version},
              "flagged": bool, "twin_created_at" } ] # summary shape for the table
}
```

GET /twins/{twin_id}

Trigger: Dashboard row action button ("View" or "Review") -> Twin Detail page.

```
Response 200: DigitalTwin (Section 9.4), plus an embedded "patient": PatientContext object
Response 404: {"detail": "Twin not found"}
```

POST /twins/{twin_id}/review

Trigger: Compliance/Governance user clicks Approve or Override on Twin Detail. Requires X-User-Role: compliance_governance header; Clinician role receives 403.

```
Request: {"reviewer_username": str, "decision": "approve"|"override", "notes": str}
```



```
-- notes is REQUIRED (min length 1) when decision == 'override', optional for 'approve'
Response 200: DigitalTwin with "review" populated
Response 400: {"detail": "Override requires a non-empty reason."}
Response 403: {"detail": "Only Compliance/Governance may review twins."}
Response 409: {"detail": "This twin has already been reviewed."}
```

10.4 Rulebook

GET /rulebook

Response 200: ComplianceRulebook (Section 9.3)

PUT /rulebook

Trigger: Compliance/Governance clicks Save Changes. Requires X-User-Role: compliance_governance.

```
Request: ComplianceRulebook + {"changed_by": str}
Response 200: {"success": true, "rulebook": ComplianceRulebook}
Response 403: {"detail": "Only Compliance/Governance may edit the rulebook."}
Notes: writes one rulebook_changelog row per changed field, e.g.
      "Changed confidence threshold 0.70 -> 0.75"
```

GET /rulebook/changelog

```
Query params: limit (default 20)
Response 200: {"entries": [{"changed_by", "change_description", "changed_at"}]}
```

10.5 Trust Monitoring

GET /trust-monitoring/summary

```
Query params: model_version (optional), from (date), to (date)
Response 200: {
  "confidence_trend": [{"date": str, "avg_confidence": float}],
  "flag_rate_trend": [{"date": str, "pct_flagged": float}],
  "avg_confidence_latest": float, "avg_confidence_delta_pct": float,
  "pct_flagged_latest": float, "pct_flagged_delta_pp": float,
  "total_predictions_in_range": int,
  "drift_status": "Drift Detected"|"Normal", "drift_since": str|null
}
```

POST /trust-monitoring/run-now

Trigger: "Run trust check now" button. Runs the same logic as the scheduled job, immediately, and returns the refreshed summary.

Response 200: same shape as GET /trust-monitoring/summary, plus "new_alerts": [DriftAlert]

GET /trust-monitoring/alerts

```
Query params: limit (default 20)
Response 200: {"alerts":
  [{"id", "alert_type", "message", "model_version", "severity", "status", "detected_at"}]}
```

10.6 Dashboard Summary

GET /dashboard/summary

Trigger: Dashboard page load. Powers the four top metric cards.

```
Response 200: {
  "total_predictions": int, "flagged_count": int, "flagged_pct": float,
  "avg_confidence": float, "active_drift_alerts": int
}
```

10.7 Patients

GET /patients/{patient_id}

```
Response 200: PatientContext
Response 404: {"detail": "Unknown patient_id"}
```

11. LangGraph Agent Architecture

The agent code structure below follows the same pattern as the reference `compliance_agent.py` provided: a node function with a docstring header, a `_build_prompt` helper, explicit retry/completeness logic where an LLM is involved, and consistent save-to-disk-or-DB of prompts/responses for auditability. The key architectural difference from the reference: in THIS system, only ONE of the four agents calls an LLM at all (`explainability_agent`). The other three are fully deterministic Python, which is a deliberate compliance-motivated design choice — see Section 15.

11.1 backend/agents/state.py

```
"""
backend/agents/state.py
Shared state object threaded through all four LangGraph nodes.
"""
from typing import TypedDict, Optional

class AgentState(TypedDict):
    patient_id: str
    prediction_input: dict          # raw PredictionInput as dict
    patient_context: dict          # PatientContext as dict
    rulebook: dict                 # current ComplianceRulebook as dict
    lineage: Optional[dict]
    compliance: Optional[dict]
    explanation: Optional[dict]    # {text, explanation_type, grounded_in_sources,
    low_grounding_confidence}
    twin: Optional[dict]          # final assembled twin (set by twin_assembler_agent)
    error: Optional[str]
```

11.2 backend/agents/graph.py

```
"""
backend/agents/graph.py
"""
from langgraph.graph import StateGraph, END
from backend.agents.state import AgentState
from backend.agents.lineage_agent import lineage_node
from backend.agents.compliance_agent import compliance_node
from backend.agents.explainability_agent import explainability_node
from backend.agents.twin_assembler_agent import twin_assembler_node

def build_graph():
    graph = StateGraph(AgentState)
    graph.add_node('lineage', lineage_node)
    graph.add_node('compliance', compliance_node)
    graph.add_node('explainability', explainability_node)
    graph.add_node('twin_assembler', twin_assembler_node)

    graph.set_entry_point('lineage')
    graph.add_edge('lineage', 'compliance')
    graph.add_edge('compliance', 'explainability')
    graph.add_edge('explainability', 'twin_assembler')
    graph.add_edge('twin_assembler', END)
    return graph.compile()

# Module-level, compiled once at import time
trust_fabric_graph = build_graph()

def run_trust_fabric_pipeline(prediction_input: dict, patient_context: dict, rulebook: dict) -> dict:
    initial_state: AgentState = {
        'patient_id': prediction_input['patient_id'],
        'prediction_input': prediction_input,
        'patient_context': patient_context,
```

```

        'rulebook': rulebook,
        'lineage': None, 'compliance': None, 'explanation': None,
        'twin': None, 'error': None,
    }
    final_state = trust_fabric_graph.invoke(initial_state)
    return final_state['twin']

```

This graph is deliberately linear — no conditional edges, no cycles. All four nodes run every time, for every prediction, regardless of outcome, because a Digital Compliance Twin must always contain lineage + compliance + explanation, whether or not the prediction ends up flagged. The only conditional logic in the whole system is downstream of the graph: whether a completed twin also appears in the Human Review Queue (a simple WHERE flagged = true filter, not a graph branch).

11.3 Agent Specifications

11.3.a Lineage Agent — backend/agents/lineage_agent.py

Purpose: record exactly which patient data and model version produced this prediction. Fully deterministic — no LLM.

Input state fields used	Output state field set
prediction_input, patient_context	lineage
<pre> """ backend/agents/lineage_agent.py Deterministic. No LLM. Records provenance only. """ from datetime import datetime, timezone from backend.agents.state import AgentState def lineage_node(state: AgentState) -> AgentState: pred = state['prediction_input'] lineage = { 'source': pred['model_name'], 'input_captured_at': pred['timestamp'], 'patient_id': pred['patient_id'], 'model_version': pred['model_version'], } return {**state, 'lineage': lineage} </pre>	

11.3.b Compliance Agent — backend/agents/compliance_agent.py

Purpose: check the prediction against the current rulebook. Fully deterministic — no LLM, unlike the reference example's compliance agent (which used llm_fast for a regulatory-narrative style review). This system's Compliance Agent is pure rule-matching by design: compliance pass/fail must be reproducible and auditable, and must never vary between runs of the same input.

Input state fields used	Output state field set
prediction_input, rulebook	compliance
<pre> """ backend/agents/compliance_agent.py Deterministic rule engine. No LLM. """ from backend.agents.state import AgentState </pre>	

```

def compliance_node(state: AgentState) ->
AgentState:
    pred = state['prediction_input']
    rulebook = state['rulebook']
    reasons = []

    version_ok = pred['model_version'] in
rulebook['approved_model_versions']
    if not version_ok:
        reasons.append(
            f"model_version
'{pred['model_version']}' not in approved
list "

f"{rulebook['approved_model_versions']}"
        )

    confidence_ok = pred['confidence'] >=
rulebook['minimum_confidence_threshold']
    if not confidence_ok:
        reasons.append(
            f"confidence {pred['confidence']}
below threshold "

f"{rulebook['minimum_confidence_threshold']}"
        )

    high_risk = pred['prediction'] in
rulebook['high_risk_conditions_requiring_revi
ew']
    if high_risk:
        reasons.append(
            f"prediction
'{pred['prediction']}' is a high-risk
condition "

            "requiring human review
regardless of confidence"
        )

    flagged = (not version_ok) or (not
confidence_ok) or high_risk

    compliance = {
        'model_version_approved': version_ok,
        'confidence_ok': confidence_ok,
        'high_risk_requires_review':
high_risk,
        'flagged': flagged,
        'flag_reasons': reasons,
    }
    return {**state, 'compliance':
compliance}

```

11.3.c Explainability Agent — backend/agents/explainability_agent.py

Purpose: generate a plain-English clinical rationale for the prediction, using RAG so the LLM can ONLY draw on curated, retrievable facts rather than its own parametric medical knowledge. This is the ONLY agent that uses an LLM. Structure below follows the reference compliance_agent.py pattern: dedicated prompt builder, retry-with-completeness-check loop, a deterministic fallback if the LLM call fails outright, and explicit saving of the prompt and response for audit purposes.

Input state fields used	Output state field set
prediction_input, patient_context (via rag_service.retrieve)	explanation

```

"""
backend/agents/explainability_agent.py
-----
--
The only LLM-calling agent in this system.
Uses llm_explainability from
backend/llm_config.py (OpenRouter,
default model openai/gpt-oss-20b:free).

RAG grounding: retrieves top_k=2 facts from
the local knowledge base
(backend/services/rag_service.py) and
instructs the model to use ONLY
those facts. Never uses SHAP/LIME -- the
underlying diagnostic model is
a black box with no accessible internals.

Required output: a short narrative string. No
section-header format is
required here (unlike the reference
compliance_agent), since this agent
produces prose for a clinician to read, not a
structured multi-field review.

Auto-retries once if the response is empty or
under 20 characters.
"""
from backend.agents.state import AgentState
from backend.llm_config import
llm_explainability
from backend.services import rag_service

AGENT_NAME = 'explainability_agent'
LOW_GROUNDING_THRESHOLD = 0.40
MAX_RETRIES = 1

def explainability_node(state: AgentState) ->
AgentState:
    pred = state['prediction_input']
    patient = state['patient_context']

    retrieved = rag_service.retrieve(
        prediction_label=pred['prediction'],
        symptoms=patient['symptoms'],
        top_k=2,
    )
    prompt = _build_prompt(pred, patient,
retrieved)

    response_text = ''
    for attempt in range(MAX_RETRIES + 1):
        try:
            response =
llm_explainability.invoke(prompt)
            response_text =
response.content.strip()
        except Exception:
            response_text =
_fallback_explanation(pred, patient)
            break
        if len(response_text) >= 20 or
attempt == MAX_RETRIES:
            break

    best_score = max((r['similarity_score']
for r in retrieved), default=0.0)
    explanation = {

```

```

        'text': response_text,
        'explanation_type':
'narrative_llm_rag_grounded',
        'grounded_in_sources': retrieved,
        'low_grounding_confidence':
best_score < LOW_GROUNDING_THRESHOLD,
    }
    return {**state, 'explanation':
explanation}

def _build_prompt(pred: dict, patient: dict,
retrieved: list) -> str:
    facts_text = '\n'.join(f"- {f['text']}"
(source: {f['source']})" for f in retrieved)
    return f"""\You are assisting a clinician
in understanding an AI diagnostic prediction.

Use ONLY the clinical facts listed below and
the patient's data. Do not
state any clinical claim not directly
supported by these facts. If the
patient's symptoms are not well explained by
the facts, say so explicitly
rather than inventing a connection. This is a
supporting explanation, not
a diagnosis.

Retrieved clinical facts:
{facts_text}

Patient: age {patient['age']], symptoms: {'
'.join(patient['symptoms'])}.
History: {patient['relevant_history']}.
Model prediction: {pred['prediction']],
confidence {pred['confidence']}.

Write a short, plain-English clinical
rationale (2-4 sentences)."""

def _fallback_explanation(pred: dict,
patient: dict) -> str:
    return (
        f"The model's
{pred['confidence']*100:.1f}% confidence "
        f"{pred['prediction'].lower()}
reading is based on the patient's "
        f"presenting profile. The
explainability service was unavailable; "
        f"this is a deterministic fallback
message, not an LLM-grounded explanation."
    )

```

11.3.d Twin Assembler Agent — `backend/agents/twin_assembler_agent.py`

Purpose: merge lineage + compliance + explanation into one final Digital Compliance Twin record. Fully deterministic — no LLM.

Input state fields used	Output state field set
lineage, compliance, explanation, patient_id, prediction_input	twin
""" backend/agents/twin_assembler_agent.py	

```

Deterministic merge. No LLM. Also generates
the twin_id.
"""
import uuid
from datetime import datetime, timezone
from backend.agents.state import AgentState

def twin_assembler_node(state: AgentState) ->
AgentState:
    pred = state['prediction_input']
    exp = state['explanation']
    twin = {
        'twin_id': str(uuid.uuid4()),
        'patient_id': state['patient_id'],
        'prediction': {
            'label': pred['prediction'],
            'confidence': pred['confidence'],
            'model_version':
pred['model_version'],
        },
        'lineage': state['lineage'],
        'compliance': state['compliance'],
        'explanation': exp['text'],
        'explanation_type':
exp['explanation_type'],
        'grounded_in_sources':
exp['grounded_in_sources'],
        'low_grounding_confidence':
exp['low_grounding_confidence'],
        'flagged':
state['compliance']['flagged'],
        'twin_created_at':
datetime.now(timezone.utc).isoformat(),
        'review': None,
    }
    return {**state, 'twin': twin}

```

11.3.e Trust Monitoring — backend/trust_monitoring/monitor.py (NOT a LangGraph node)

Purpose: periodically scan stored twins for confidence drift and rising flag rate. Deliberately implemented OUTSIDE the LangGraph pipeline, as a standalone scheduled function, because it operates on many twins at once (a batch/aggregate computation) rather than processing one prediction through a sequential agent chain. Registered with APScheduler in backend/main.py on startup, default interval 6 hours, also callable synchronously from POST /trust-monitoring/run-now.

```

"""
backend/trust_monitoring/monitor.py
"""
CONFIDENCE_DROP_THRESHOLD_PCT = 15.0    # alert if avg confidence falls >= 15% over the window
FLAG_RATE_THRESHOLD_PCT = 10.0          # alert if % flagged exceeds 10%
DEFAULT_WINDOW_DAYS = 7

def run_trust_check(model_version: str | None = None, window_days: int = DEFAULT_WINDOW_DAYS) ->
dict:
    # 1. Query digital_twins grouped by day for the window, optionally filtered
    #    by model_version, computing avg(prediction.confidence) and
    #    pct flagged per day.
    # 2. Compare the first day's avg confidence to the last day's.
    #    If the percentage drop >= CONFIDENCE_DROP_THRESHOLD_PCT, write a
    #    drift_alerts row: alert_type='confidence_drop', severity='High'.
    # 3. If the latest day's pct flagged > FLAG_RATE_THRESHOLD_PCT, write a
    #    drift_alerts row: alert_type='flag_rate_rise', severity='High'.
    # 4. If neither condition triggers, write (or update the existing
    #    unresolved) freshness_ok row with severity='Low', status='Cleared'.
    # 5. Return the same summary shape as GET /trust-monitoring/summary.

```


...

11.4 Human Review Queue — Not an Agent

Important scope clarification: there is no separate "Review Queue Agent" and no separate database table for it. The Human Review Queue is simply the result of `GET /twins?status=flagged` — a query-time filter on the `digital_twins` table (`WHERE flagged = true AND review IS NULL`). Modeling this as an LLM agent would add complexity with no benefit, since routing to review is a deterministic consequence of the Compliance Agent's output, not a judgment call requiring reasoning.

12. RAG Knowledge Base Specification

data/knowledge_base.json contains curated, source-attributed clinical facts. This is NOT patient data and NOT prediction data — it is a small, hand-authored reference set the Explainability Agent is constrained to draw from.

12.1 Schema

```
[
  {
    "id": "kb001",
    "condition": "Pneumonia",
    "text": "Common presenting symptoms of pneumonia include fever, cough (often productive), shortness of breath, chest pain, fatigue, and low oxygen saturation. Risk factors include smoking history, immunocompromised status, recent hospitalization, and advanced age.",
    "source": "CDC Pneumonia Clinical Overview"
  }
]
```

12.2 Minimum Required Entries

At least the following entries must exist, covering every value that can appear in prediction and rulebook fields:

id	condition	Summary of text
kb001	Pneumonia	General presenting symptoms + risk factors (fever, cough, SOB, chest pain, fatigue, low SpO2; smoking, immunocompromised, recent hospitalization, age)
kb002	Pneumonia	Atypical presentation in elderly patients — confusion/altered mental status without classic respiratory symptoms
kb003	Tumor	General radiological indicators and risk-factor context for a mass/tumor finding
kb004	Cancer	General risk-factor context (age, smoking history, prior history) relevant to a cancer-flagged finding
kb005	Normal	A normal chest X-ray finding indicates no visible signs of pneumonia, mass, or significant abnormality; some patients with normal imaging may still present mild, non-specific symptoms unrelated to respiratory infection

12.3 Retrieval Logic — backend/services/rag_service.py

```
"""
backend/services/rag_service.py
Local embedding model -- no external embedding API call.
"""
from sentence_transformers import SentenceTransformer
import numpy as np, json

_embedder = SentenceTransformer('all-MiniLM-L6-v2')
_knowledge_base = []
_kb_embeddings = None

def load_knowledge_base(path='data/knowledge_base.json'):
    global _knowledge_base, _kb_embeddings
    with open(path) as f:
        _knowledge_base = json.load(f)
    _kb_embeddings = _embedder.encode([e['text'] for e in _knowledge_base])
```

```
def retrieve(prediction_label: str, symptoms: list[str], top_k: int = 2) -> list[dict]:
    query = f"{prediction_label}: {' '.join(symptoms)}"
    query_vec = _embedder.encode([query])[0]
    scores = np.dot(_kb_embeddings, query_vec) / (
        np.linalg.norm(_kb_embeddings, axis=1) * np.linalg.norm(query_vec)
    )
    top_idx = np.argsort(scores)[::-1][:top_k]
    return [
        {**_knowledge_base[i], 'similarity_score': float(scores[i])}
        for i in top_idx
    ]
```

`load_knowledge_base()` is called once, in `backend/main.py`'s startup event. Embeddings are cached in the module-level `_kb_embeddings` variable for the lifetime of the process — they are never recomputed per-request.

12.4 Low-Grounding Signal

If the best-matching KB entry's `similarity_score` is below 0.40, the twin's `low_grounding_confidence` field is set to `true`. This is a visible signal in the UI (Twin Detail, Explanation card) that the explanation may be weaker than usual — the retrieval did not find a strong match for this case. This does NOT automatically flag the twin for compliance review; it is informational only, distinct from the `compliance.flagged` field.

13. Frontend Architecture and Complete UI Specification

This section describes every page exactly as shown in the provided UI screenshots, adapted to this system's real data (model name vit-xray-pneumonia-classification, conditions Pneumonia/Tumor/Cancer, versions v2.0/v2.1, threshold 0.75) rather than the illustrative example values (SepsisRiskClassifier, HeartFailureRisk, etc.) that appear in the reference screenshots. The screenshots establish visual structure and component behavior; the data displayed must come from this system's canonical schemas (Section 9).

13.1 Global Layout

- Two-panel authenticated layout: fixed left Sidebar + flexible main content area. Login page is the one exception — full-viewport two-column, no sidebar.
- Top bar (all authenticated pages): app name + shield icon left, an organization selector dropdown showing a single static value "Global Health System" (non-functional in phase 1 — present for future multi-tenant extensibility, not wired to any backend filter), and a user chip on the right showing avatar initials, display name, and role label.
- Sidebar renders navigation items conditionally by role: Clinician sees only Dashboard. Compliance/Governance sees Dashboard, Rulebook Settings, and Trust Monitoring.
- Design tokens: dark navy sidebar (#0A1628-class), white content background, status colors: green=Cleared/Pass, amber=Flagged/Warning, red=Error/Override, blue=primary actions.

13.2 Page 1 — Login (/login)

Full-viewport two-column layout, no sidebar.

Left panel (hero, dark background)

- Large heading "Trust Fabric"
- Subheading tag "DIGITAL COMPLIANCE TWIN // v1"
- Tagline: "Every AI clinical prediction, automatically explained, checked, and made defensible."
- Four feature rows, each with an icon, title, and one-line description: Full Lineage ("Trace every prediction back to its source data and model version"), Compliance Built-In ("Automatic checks against approved versions and thresholds"), Plain-English Explanations ("AI reasoning translated into clinical language"), Continuous Trust Monitoring ("Drift and confidence tracked over time, not just at prediction time")

Right panel (sign-in card)

- Label "Authenticate to continue", heading "Sign In"
- Error banner (conditionally rendered): red-bordered box, "Invalid credentials" — shown only after a failed login attempt
- Username/ID text input, Password input with a show/hide eye-icon toggle
- "Forgot password?" link — renders a toast "Not available in this phase" on click, no backend endpoint
- Sign In button (primary, full width) — calls POST /auth/login
- Footer note: "Role is resolved automatically from your credentials."

Behavior

- On success: store {username, role, display_name} in AuthContext + sessionStorage, redirect to /dashboard
- On 401: show the Invalid credentials banner, clear the password field, keep username

13.3 Page 2 — Dashboard (/dashboard)

Two visual variants of the same page: Clinician sees a collapsed/minimal sidebar (icon rail only, Dashboard highlighted); Compliance/Governance sees the full sidebar with all three nav labels. The content area is otherwise identical for both roles except the row action button label.

Top metric cards (4, from GET /dashboard/summary)

Card	Value source
Total Predictions	total_predictions, subtext "All time"
Flagged Count	flagged_count, subtext "{flagged_pct}% of total"
Avg Confidence	avg_confidence, subtext "Across all predictions"
Drift Alerts	active_drift_alerts, subtext "Active alerts"

Filter bar

- Status dropdown: All Statuses / Cleared / Flagged -> maps to GET /twins?status=
- Model Version dropdown: All Versions / v2.0 / v2.1 (populated from distinct values actually present) -> maps to ?model_version=
- Patient ID search box (debounced) -> maps to ?search=
- Reset Filters button clears all three and re-fetches

Predictions table

- Columns: Patient ID | Model + Version | Prediction | Confidence | Status badge | Action
- Status badge: "Cleared" (green) when flagged=false; "Flagged" (amber) when flagged=true
- Action button: "View" for cleared rows, "Review" for flagged rows — both navigate to /twins/:twin_id
- Column headers for Patient ID / Model+Version / Prediction / Confidence are clickable and toggle sort_by/sort_dir
- Pagination footer: "Showing X to Y of Z results" + numbered page controls, from the total/page/limit fields of GET /twins

13.4 Page 3 — Twin Detail (/twins/:twin_id)

Two role-based variants, driven by the same GET /twins/{twin_id} response: Clinician sees a read-only page; Compliance/Governance sees the same page plus a Compliance Action panel at the bottom.

Header

- "Back to Dashboard" link — returns to Dashboard, preserving prior filter/pagination state via query params
- "Twin Detail" title

Summary bar (4 fields, top of page)

Field	Source
Prediction	twin.prediction.label, with a colored icon (amber warning icon if flagged, else neutral)
Confidence	twin.prediction.confidence, shown both as decimal and percentage
Model + Version	twin.prediction.model_version + twin.lineage.source, subtext "Clinical AI Model"
Time Generated	twin.twin_created_at, plus a relative "(N hours ago)" computed client-side

Four content cards

1. Lineage — Input Source (twin.lineage.source), Data Capture Time (twin.lineage.input_captured_at), a static three-step "Data Pipeline" label ("Ingestion -> Validation -> Model Inference", descriptive only, not a live status), and the twin_id shown as "Data Snapshot ID".

2. Explanation — badge reading "AI-generated narrative" (this is the honest, generic UI label; the underlying explanation_type value 'narrative_llm_rag_grounded' is available via a tooltip/info icon for technically-inclined users), the explanation text (twin.explanation), and, if grounded_in_sources is non-empty, a small "Grounded in:" line listing each source's source field. If low_grounding_confidence is true, render an additional small amber note: "This explanation has lower supporting evidence than usual."

3. Compliance — a Rule | Status two-column mini-table. Because this system's canonical rulebook (Section 9.3) defines exactly three checks, this card renders exactly three rows plus an overall status row, NOT the seven illustrative rows shown in the reference screenshot (which depict a different, hypothetical rulebook):

Rule	Status source
Model Version Approved	compliance.model_version_approved -> Pass/Fail
Confidence Threshold Met	compliance.confidence_ok -> Pass/Fail
High-Risk Condition Review	compliance.high_risk_requires_review -> "Required" (amber) / "Not Required" (neutral)
Overall Status	compliance.flagged -> "Flagged for Review" (amber) / "Cleared" (green)

If compliance.flagged is true, list each string in compliance.flag_reasons underneath the table as supporting detail text.

4. Patient Context — Patient ID, Age, Primary Symptoms (comma-joined list), Relevant History, and Scan Type, sourced from the embedded patient object in the GET /twins/{twin_id} response.

Footer / Compliance Action panel

- Clinician view: a single line, "Viewed by {display_name} on {current timestamp}" — this is a client-side display only, NOT written to the audit log (viewing is not an auditable action in phase 1)
- Compliance/Governance view, when twin.flagged is true and twin.review is null: a "Compliance Action" panel with an Approve button (green), an Override button (red), and a notes textarea labeled "Override requires a reason (required)" with a live character counter (0/1000). The Approve button is enabled with an empty notes field; the Override button is disabled until notes.length > 0.
- Once twin.review is populated (either decision), the action panel is replaced with a footer line: "Reviewed by {review.reviewed_by} on {review.reviewed_at}", and the decision + notes are shown read-only above it.
- If twin.flagged is false, no action panel is shown for either role — there is nothing to review.

13.5 Page 4 — Rulebook Settings (/rulebook) — Compliance/Governance only

Route does not exist in navigation for Clinician; direct navigation attempts redirect to /dashboard.

- Breadcrumb: Dashboard > Rulebook Settings, with a "Compliance/Governance only" badge next to the page title
- Toast notification (top-right, green, auto-dismiss): "Settings saved successfully." after a successful PUT /rulebook

Approved Model Versions card

- Tag/chip input showing each string in approved_model_versions with an X to remove; a text field with placeholder "Add a model version and press Enter" to add new ones

Confidence Threshold card

- A numeric input (bound to the same value as the slider) plus a 0.0-1.0 range slider; label "Set the minimum confidence score required for predictions to be auto-cleared."; range label "Range: 0.0 - 1.0"

High-Risk Conditions card

- Tag/chip input showing each string in `high_risk_conditions_requiring_review` (Pneumonia, Tumor, Cancer) with the same add/remove pattern as Approved Model Versions

Save Changes button

- Bottom-right, primary. Calls `PUT /rulebook` with the full edited object + `{"changed_by": current_username}`

Change Log table

- Columns: User (avatar initials + name) | Change (`change_description`) | Date & Time (`changed_at`), from `GET /rulebook/changelog`
- "View full change history" link at the bottom (renders the same data with limit removed / higher limit — no separate endpoint needed)

13.6 Page 5 — Trust Monitoring (/trust-monitoring) — Compliance/Governance only

- Breadcrumb: Dashboard > Trust Monitoring, with a "Compliance/Governance only" badge

Drift alert banner (conditionally rendered, amber)

- Shown when `drift_status` is "Drift Detected": title "Drift threshold crossed", the triggering alert's message, "Detected on {`detected_at`}", and a "View Drift Details" button that scrolls to / highlights the Recent Drift Alerts table

Filter row

- Model / Version dropdown (populated from distinct `model_version` values present in twins)
- Date Range picker (defaults to the last 7 days)
- "Run trust check now" button (top-right) — calls `POST /trust-monitoring/run-now`, shows a brief loading state, then refreshes all charts/cards/table on the page

Two trend charts (Recharts LineChart)

- Confidence Trend — average confidence per day, with a dashed reference line at the rulebook's `minimum_confidence_threshold`, a Daily/Weekly toggle, and a hover tooltip showing the exact date/value and delta vs. the range start
- Flag-Rate Trend — percentage of predictions flagged per day, with a dashed reference line at the 10% drift threshold, same toggle and tooltip pattern

Four summary stat cards

Card	Source
Avg Confidence (latest day)	<code>avg_confidence_latest</code> , delta vs range start (<code>avg_confidence_delta_pct</code>)
% Flagged (latest day)	<code>pct_flagged_latest</code> , delta vs range start (<code>pct_flagged_delta_pp</code>)
Predictions	<code>total_predictions_in_range</code> , subtext "Total in range"
Drift Status	<code>drift_status</code> badge + "Since { <code>drift_since</code> }" if Drift Detected

Recent Drift Alerts table

- Columns: Alert (message) | Model / Version | Detected On | Severity badge (High/Low) | Status badge (Active/Cleared), from `GET /trust-monitoring/alerts`
- "View all alerts" link at the bottom (same endpoint, higher limit)

Footer

- "Most recent trust check: {timestamp of the last run}" and "Auto-checks enabled every 6 hours" (matches the APScheduler interval configured in backend/main.py)

14. Authentication and Authorization

14.1 backend/services/auth_service.py

```
USERS = {
    'dr.mitchell': {
        'password': 'clinician123',    # plaintext -- see 14.3 limitation note
        'role': 'clinician',
        'display_name': 'Dr. Sarah Mitchell',
    },
    'compliance.lead': {
        'password': 'compliance123',
        'role': 'compliance_governance',
        'display_name': 'Compliance Team',
    },
}

def authenticate(username: str, password: str) -> dict | None:
    user = USERS.get(username)
    if user and user['password'] == password:
        return {'username': username, 'role': user['role'], 'display_name': user['display_name']}
    return None
```

14.2 Authorization Mechanism

Since there is no JWT, the frontend sends the currently logged-in user's role on every request that requires authorization, as a plain header: `X-User-Role: clinician` or `X-User-Role: compliance_governance`. A small FastAPI dependency, `require_role('compliance_governance')`, reads this header and raises HTTP 403 if it does not match. This guards `POST /twins/{id}/review` and `PUT /rulebook`.

14.3 Honest Limitation (must be stated in code comments, not hidden)

This mechanism is NOT cryptographically secure — a client could send any `X-User-Role` value it wants, since nothing is signed. This is an explicit, documented simplification for phase 1, matching the instruction to skip JWT/Keycloak for now. `backend/routers/twins.py` and `rulebook.py` must include a comment block stating this limitation plainly, and Section 20 (Future Extension Points) lists JWT + Keycloak as the direct upgrade path.

15. Business Rules

15.1 Compliance Rules (enforced by `compliance_agent.py`)

- A prediction is flagged if ANY of: (a) `model_version` is not in `rulebook.approved_model_versions`, (b) `confidence < rulebook.minimum_confidence_threshold`, (c) prediction is in `rulebook.high_risk_conditions_requiring_review`
- All three checks always run and are always recorded in `compliance.flag_reasons`, even if only one caused the flag — this gives Compliance/Governance the full picture, not just the first failure
- Flagging is binary and deterministic — there is no partial-flag or severity gradient at the compliance-check level (severity IS used at the drift-alert level, Section 15.3)

15.2 Human Review and Override Workflow

- Only twins where `flagged = true` and review IS NULL appear in the Human Review Queue view (GET `/twins?status=flagged`)
- Approve: reviewer confirms the AI's prediction stands; notes are optional
- Override: reviewer disagrees with or corrects the AI's finding; notes are REQUIRED (see Section 10.3) and must describe the reason
- A twin can be reviewed exactly once — POST `/twins/{id}/review` on an already-reviewed twin returns HTTP 409
- Only the Compliance/Governance role may review; Clinician requests to this endpoint return HTTP 403
- Reviewing a twin writes an `audit_log` entry (`action='twin_reviewed'`) in addition to updating the twin's review field

15.3 Drift Detection Rules (`drift_service.py` / `trust_monitoring/monitor.py`)

Rule	Condition	Resulting alert
Confidence drop	avg confidence on the last day of the window is $\geq$ 15% lower than the first day of the window, for a given <code>model_version</code>	<code>alert_type='confidence_drop'</code> , <code>severity='High'</code> , <code>status='Active'</code>
Flag-rate rise	% of predictions flagged on the latest day exceeds 10%	<code>alert_type='flag_rate_rise'</code> , <code>severity='High'</code> , <code>status='Active'</code>
Freshness / normal	neither condition above is true	<code>alert_type='freshness_ok'</code> , <code>severity='Low'</code> , <code>status='Cleared'</code> (upserted, not duplicated)

Default window: 7 days (`DEFAULT_WINDOW_DAYS` in `monitor.py`). Both thresholds (15% and 10%) are named constants at the top of `monitor.py`, not hardcoded inline, so they can be tuned without touching the alerting logic.

15.4 Confidence Rules

- Confidence is always stored and displayed as a 0.0-1.0 float; the UI renders it both as a decimal (e.g. 0.8423) and a percentage (e.g. 84.2%) depending on context — Dashboard table shows the decimal, Twin Detail summary bar shows both
- The `minimum_confidence_threshold` from the rulebook is the ONLY confidence rule that affects compliance flagging; the drift thresholds in 15.3 are separate and affect only Trust Monitoring alerts, never an individual twin's flagged field

16. Audit Logging

Every one of the following actions writes exactly one row to `audit_log` (Section 8.6). No action may be silently skipped, and no existing row may ever be updated or deleted by application code.

Action	entity_type	actor	details example
Twin created (every POST /predictions)	twin	system	{model_version, flagged}
Twin reviewed (approve or override)	twin	reviewer username	{decision, notes}
Rulebook updated	rulebook	changed_by username	{changed_fields: [...]}
Drift alert raised	drift_alert	system	{alert_type, model_version, severity}

17. Configuration Files

17.1 .env.example

```
DATABASE_URL=postgresql://trust_fabric:trust_fabric_dev_password@localhost:5432/trust_fabric
OPENROUTER_API_KEY=sk-or-...
EXPLAINABILITY_MODEL=openai/gpt-oss-20b:free
TRUST_MONITORING_INTERVAL_HOURS=6
CORS_ALLOWED_ORIGIN=http://localhost:5173
```

17.2 config.yaml

```
drift:
  confidence_drop_threshold_pct: 15.0
  flag_rate_threshold_pct: 10.0
  default_window_days: 7
rag:
  low_grounding_threshold: 0.40
  top_k: 2
  embedding_model: all-MiniLM-L6-v2
```

17.3 frontend/.env.local

```
VITE_API_URL=http://localhost:8000
```

18. Docker and Docker Compose

Scope decision: only PostgreSQL is containerized. FastAPI, the LangGraph agents, APScheduler, and the React frontend all run directly on the host during development — this matches the project's general principle of not introducing infrastructure the current scale does not need. See the delivered `docker-compose.yml` for the exact single-service definition (postgres:16-alpine, healthcheck, named volume).

- Start: `docker compose up -d`
- The healthcheck (`pg_isready`) must pass before `scripts/init_db.py` is run
- No Dockerfile is required for the backend or frontend in phase 1 — they are run with `uvicorn` and `npm run dev` respectively

19. Testing Strategy

19.1 Unit Tests (tests/)

File	Covers
test_lineage_agent.py	lineage_node produces the exact expected dict shape from a fixed input
test_compliance_agent.py	each of the 3 rules independently: approved-version pass/fail, threshold pass/fail, high-risk pass/fail; combinations; flag_reasons content
test_explainability_agent.py	prompt construction includes retrieved facts; fallback path triggers on a mocked LLM exception; low_grounding_confidence flips correctly at the 0.40 boundary
test_twin_assembler_agent.py	final twin dict contains all required fields with correct types
test_rulebook_rules.py	updating the rulebook writes the correct changelog description string
test_drift_service.py	15% confidence-drop and 10% flag-rate thresholds trigger alerts at the boundary and not below it

19.2 Integration Tests

- test_api_predictions.py — POST /predictions end-to-end through the real (test) database, asserting the response twin matches the four-agent output for a known fixture
- test_api_twins.py — GET /twins filtering/sorting/pagination; POST /twins/{id}/review success, 403, 409, and missing-notes-on-override 400 cases

19.3 End-to-End (manual, scripted)

- scripts/run_pipeline_on_seed_data.py doubles as an E2E smoke test: all 60 seed predictions must process without error and produce the expected ~24 flagged twins (matching the uploaded human_review_queue.json reference count)

19.4 Acceptance Criteria

- A prediction for an approved model version, above the confidence threshold, and not a high-risk condition, is never flagged
- A prediction for 'Pneumonia' is always flagged, regardless of confidence or model version, per the high_risk_conditions_requiring_review rule
- Every twin has a non-null explanation, explanation_type, and grounded_in_sources — the Explainability Agent never returns an empty result silently
- Clinician role can never successfully call POST /twins/{id}/review or PUT /rulebook (403 in both cases)
- An Override action with an empty notes field is rejected client-side (button disabled) AND server-side (400) — both layers must enforce this independently
- Running the same prediction through the pipeline twice produces identical lineage and compliance results (fully deterministic); only the explanation text may vary slightly (LLM-generated) and grounded_in_sources must reference the same KB entries both times

20. Future Extension Points

Area	Phase 1 (this PRD)	Production upgrade path
Diagnostic model	Mock JSON predictions	Real model-serving endpoint (Triton / Azure ML / vendor API) behind the same PredictionInput schema
Auth	Hardcoded username/password, X-User-Role header	JWT + Keycloak, matching the pattern used in other internal platforms
Ingestion	Synchronous HTTP POST	Message queue (Kafka/RabbitMQ) if prediction volume grows beyond low hundreds
Scheduling	In-process APScheduler	Airflow, once there are multiple interdependent scheduled jobs
Explainability LLM	OpenRouter openai/gpt-oss-20b:free	openai/gpt-4o or Azure OpenAI, via the same llm_config.py swap point
Multi-model support	Single model_name assumed throughout examples	Per-model rulebooks and per-model drift baselines
Audit storage	Postgres append-only table	Tamper-evident hash-chained log or dedicated immutable store
Alerting	Dashboard/Trust Monitoring page only	Real-time push (Slack/Teams webhook) on new drift_alerts rows
Patient data source	Static data/patients.json seed	Real FHIR/EHR integration

21. Startup Instructions

See the accompanying README.md for the full, copy-pasteable setup sequence. Summary of build order for a coding assistant implementing this from scratch:

- 1. .env.example, config.yaml, requirements.txt, docker-compose.yml
- 2. backend/models/schemas.py — all Pydantic models (Section 9)
- 3. backend/db/models.py + session.py, then scripts/init_db.py
- 4. backend/services/auth_service.py, audit_service.py, rulebook_service.py
- 5. data/knowledge_base.json + backend/services/rag_service.py
- 6. backend/llm_config.py
- 7. backend/agents/state.py, then lineage_agent.py, compliance_agent.py, explainability_agent.py, twin_assembler_agent.py, then graph.py
- 8. backend/trust_monitoring/monitor.py + drift_service.py
- 9. backend/routers/* + backend/main.py
- 10. scripts/seed_data.py, scripts/run_pipeline_on_seed_data.py
- 11. tests/ — all files from Section 19
- 12. frontend/ — contexts, then pages in this order: Login, Dashboard, TwinDetail, RulebookSettings, TrustMonitoring
- 13. frontend components (Sidebar, Topbar, ProtectedRoute, ui/*, charts/*) as needed by the pages above